JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class Executors

java.lang.Object
    java.util.concurrent.Executors

```
public class Executors
extends Object
```

Factory and utility methods for `Executor`, `ExecutorService`, `ScheduledExecutorService`, `ThreadFactory`, and `Callable` classes defined in this package. This class supports the following kinds of methods:

- Methods that create and return an `ExecutorService` set up with commonly useful configuration settings.
- Methods that create and return a `ScheduledExecutorService` set up with commonly useful configuration settings.
- Methods that create and return a "wrapped" ExecutorService, that disables reconfiguration by making implementation-specific methods inaccessible.
- Methods that create and return a `ThreadFactory` that sets newly created threads to a known state.
- Methods that create and return a `Callable` out of other closure-like forms, so they can be used in execution methods requiring `Callable`.

Since:

1.5

Method Summary

Concrete Methods

Modifier and Type	Method
`static Callable<Object>`	`callable(PrivilegedAction<?> action)`

static **Callable**<**Object**>	**callable**(**PrivilegedExceptionAction**<?> action)
static **Callable**<**Object**>	**callable**(**Runnable** task)
static <T> **Callable**<T>	**callable**(**Runnable** task, T result)
static **ThreadFactory**	**defaultThreadFactory**()
static **ExecutorService**	**newCachedThreadPool**()
static **ExecutorService**	**newCachedThreadPool**(**ThreadFactory** threadFactory)
static **ExecutorService**	**newFixedThreadPool**(int nThreads)
static **ExecutorService**	**newFixedThreadPool**(int nThreads, **ThreadFactory** t

static ScheduledExecutorService	newScheduledThreadPool(int corePoolSize)
static ScheduledExecutorService	newScheduledThreadPool(int corePoolSize, ThreadF
static ExecutorService	newSingleThreadExecutor()
static ExecutorService	newSingleThreadExecutor(ThreadFactory threadFact
static ScheduledExecutorService	newSingleThreadScheduledExecutor()
static ScheduledExecutorService	newSingleThreadScheduledExecutor(ThreadFactory t
static ExecutorService	newWorkStealingPool()
static ExecutorService	newWorkStealingPool(int parallelism)

static <T> **Callable**<T>	**privilegedCallable**(**Callable**<T> callable)
static <T> **Callable**<T>	**privilegedCallableUsingCurrentClassLoader**(**Callab**
static **ThreadFactory**	**privilegedThreadFactory**()
static **ExecutorService**	**unconfigurableExecutorService**(**ExecutorService** ex
static **ScheduledExecutorService**	**unconfigurableScheduledExecutorService**(**Scheduled**

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

newFixedThreadPool

```
public static ExecutorService newFixedThreadPool(int nThreads)
```

Creates a thread pool that reuses a fixed number of threads operating off a shared unbounded queue. At any point, at most `nThreads` threads will be active processing tasks. If additional tasks are submitted when all threads are active, they will wait in the queue until a thread is available. If any thread terminates due to a failure during execution prior to shutdown, a new one will take its place if needed to execute subsequent tasks. The threads in the pool will exist until it is explicitly `shutdown`.

Parameters:

`nThreads - the number of threads in the pool`

Returns:

`the newly created thread pool`

Throws:

`IllegalArgumentException - if nThreads <= 0`

newWorkStealingPool

```
public static ExecutorService newWorkStealingPool(int parallelism)
```

Creates a thread pool that maintains enough threads to support the given parallelism level, and may use multiple queues to reduce contention. The parallelism level corresponds to the maximum number of threads actively engaged in, or available to engage in, task processing. The actual number of threads may grow and shrink dynamically. A work-stealing pool makes no guarantees about the order in which submitted tasks are executed.

Parameters:

`parallelism - the targeted parallelism level`

Returns:

`the newly created thread pool`

Throws:

`IllegalArgumentException - if parallelism <= 0`

Since:

`1.8`

newWorkStealingPool

```
public static ExecutorService newWorkStealingPool()
```

Creates a work-stealing thread pool using all `available processors` as its target parallelism level.

Returns:

the newly created thread pool

Since:

1.8

See Also:

`newWorkStealingPool(int)`

newFixedThreadPool

```
public static ExecutorService newFixedThreadPool(int nThreads,
                                                 ThreadFactory threadFactory)
```

Creates a thread pool that reuses a fixed number of threads operating off a shared unbounded queue, using the provided ThreadFactory to create new threads when needed. At any point, at most `nThreads` threads will be active processing tasks. If additional tasks are submitted when all threads are active, they will wait in the queue until a thread is available. If any thread terminates due to a failure during execution prior to shutdown, a new one will take its place if needed to execute subsequent tasks. The threads in the pool will exist until it is explicitly `shutdown`.

Parameters:

`nThreads` - the number of threads in the pool

`threadFactory` - the factory to use when creating new threads

Returns:

the newly created thread pool

Throws:

`NullPointerException` - if threadFactory is null

`IllegalArgumentException` - if nThreads <= 0

newSingleThreadExecutor

```
public static ExecutorService newSingleThreadExecutor()
```

Creates an Executor that uses a single worker thread operating off an unbounded queue. (Note however that if this single thread terminates due to

a failure during execution prior to shutdown, a new one will take its place if needed to execute subsequent tasks.) Tasks are guaranteed to execute sequentially, and no more than one task will be active at any given time. Unlike the otherwise equivalent `newFixedThreadPool(1)` the returned executor is guaranteed not to be reconfigurable to use additional threads.

Returns:

`the newly created single-threaded Executor`

newSingleThreadExecutor

```
public static ExecutorService newSingleThreadExecutor(ThreadFactory threadFactory
```

Creates an Executor that uses a single worker thread operating off an unbounded queue, and uses the provided ThreadFactory to create a new thread when needed. Unlike the otherwise equivalent `newFixedThreadPool(1, threadFactory)` the returned executor is guaranteed not to be reconfigurable to use additional threads.

Parameters:

`threadFactory - the factory to use when creating new threads`

Returns:

`the newly created single-threaded Executor`

Throws:

`NullPointerException - if threadFactory is null`

newCachedThreadPool

```
public static ExecutorService newCachedThreadPool()
```

Creates a thread pool that creates new threads as needed, but will reuse previously constructed threads when they are available. These pools will typically improve the performance of programs that execute many short-lived asynchronous tasks. Calls to `execute` will reuse previously constructed threads if available. If no existing thread is available, a new thread will be created and added to the pool. Threads that have not been used for sixty seconds are terminated and removed from the cache. Thus, a pool that remains idle for long enough will not consume any resources. Note that pools with similar properties but different details (for example, timeout parameters) may be created using `ThreadPoolExecutor` constructors.

Returns:

`the newly created thread pool`

newCachedThreadPool

```
public static ExecutorService newCachedThreadPool(ThreadFactory threadFactory)
```

Creates a thread pool that creates new threads as needed, but will reuse previously constructed threads when they are available, and uses the provided ThreadFactory to create new threads when needed.

Parameters:

`threadFactory - the factory to use when creating new threads`

Returns:

`the newly created thread pool`

Throws:

`NullPointerException - if threadFactory is null`

newSingleThreadScheduledExecutor

```
public static ScheduledExecutorService newSingleThreadScheduledExecutor()
```

Creates a single-threaded executor that can schedule commands to run after a given delay, or to execute periodically. (Note however that if this single thread terminates due to a failure during execution prior to shutdown, a new one will take its place if needed to execute subsequent tasks.) Tasks are guaranteed to execute sequentially, and no more than one task will be active at any given time. Unlike the otherwise equivalent `newScheduledThreadPool(1)` the returned executor is guaranteed not to be reconfigurable to use additional threads.

Returns:

`the newly created scheduled executor`

newSingleThreadScheduledExecutor

```
public static ScheduledExecutorService newSingleThreadScheduledExecutor(ThreadFac
```

Creates a single-threaded executor that can schedule commands to run after a given delay, or to execute periodically. (Note however that if this single thread terminates due to a failure during execution prior to shutdown, a new one will take its place if needed to execute subsequent tasks.) Tasks are guaranteed to execute sequentially, and no more than one task will be active at any given time. Unlike the otherwise equivalent `newScheduledThreadPool(1, threadFactory)` the returned executor is guaranteed not to be reconfigurable to use additional threads.

Parameters:

`threadFactory - the factory to use when creating new threads`

Returns:

a newly created scheduled executor

Throws:

NullPointerException - if threadFactory is null

newScheduledThreadPool

```
public static ScheduledExecutorService newScheduledThreadPool(int corePoolSize)
```

Creates a thread pool that can schedule commands to run after a given delay, or to execute periodically.

Parameters:

corePoolSize - the number of threads to keep in the pool, even if they are idle

Returns:

a newly created scheduled thread pool

Throws:

IllegalArgumentException - if corePoolSize < 0

newScheduledThreadPool

```
public static ScheduledExecutorService newScheduledThreadPool(int corePoolSize,
                                                               ThreadFactory threa
```

Creates a thread pool that can schedule commands to run after a given delay, or to execute periodically.

Parameters:

corePoolSize - the number of threads to keep in the pool, even if they are idle

threadFactory - the factory to use when the executor creates a new thread

Returns:

a newly created scheduled thread pool

Throws:

IllegalArgumentException - if corePoolSize < 0

NullPointerException - if threadFactory is null

unconfigurableExecutorService

```
public static ExecutorService unconfigurableExecutorService(ExecutorService execu
```

Returns an object that delegates all defined `ExecutorService` methods to the given executor, but not any other methods that might otherwise be accessible using casts. This provides a way to safely "freeze" configuration and disallow tuning of a given concrete implementation.

Parameters:

`executor - the underlying implementation`

Returns:

`an ExecutorService instance`

Throws:

`NullPointerException - if executor null`

unconfigurableScheduledExecutorService

```
public static ScheduledExecutorService unconfigurableScheduledExecutorService(Sch
```

Returns an object that delegates all defined `ScheduledExecutorService` methods to the given executor, but not any other methods that might otherwise be accessible using casts. This provides a way to safely "freeze" configuration and disallow tuning of a given concrete implementation.

Parameters:

`executor - the underlying implementation`

Returns:

`a ScheduledExecutorService instance`

Throws:

`NullPointerException - if executor null`

defaultThreadFactory

```
public static ThreadFactory defaultThreadFactory()
```

Returns a default thread factory used to create new threads. This factory creates all new threads used by an Executor in the same `ThreadGroup`. If there is a `SecurityManager`, it uses the group of `System.getSecurityManager()`, else the group of the thread invoking this `defaultThreadFactory` method. Each new thread is created as a non-daemon thread with priority set to the smaller of `Thread.NORM_PRIORITY` and the maximum priority permitted in the thread group. New threads have names accessible via `Thread.getName()` of *pool-N-thread-M*, where *N* is the

sequence number of this factory, and *M* is the sequence number of the thread created by this factory.

Returns:

`a thread factory`

privilegedThreadFactory

```
public static ThreadFactory privilegedThreadFactory()
```

Returns a thread factory used to create new threads that have the same permissions as the current thread. This factory creates threads with the same settings as `defaultThreadFactory()`, additionally setting the AccessControlContext and contextClassLoader of new threads to be the same as the thread invoking this `privilegedThreadFactory` method. A new `privilegedThreadFactory` can be created within an `AccessController.doPrivileged` action setting the current thread's access control context to create threads with the selected permission settings holding within that action.

Note that while tasks running within such threads will have the same access control and class loader settings as the current thread, they need not have the same `ThreadLocal` or `InheritableThreadLocal` values. If necessary, particular values of thread locals can be set or reset before any task runs in `ThreadPoolExecutor` subclasses using `ThreadPoolExecutor.beforeExecute(Thread, Runnable)`. Also, if it is necessary to initialize worker threads to have the same InheritableThreadLocal settings as some other designated thread, you can create a custom ThreadFactory in which that thread waits for and services requests to create others that will inherit its values.

Returns:

`a thread factory`

Throws:

`AccessControlException - if the current access control context does not have permission to both get and set context class loader`

callable

```
public static <T> Callable<T> callable(Runnable task,
                                       T result)
```

Returns a `Callable` object that, when called, runs the given task and returns the given result. This can be useful when applying methods requiring a `Callable` to an otherwise resultless action.

Type Parameters:

T - the type of the result

Parameters:

task - the task to run

result - the result to return

Returns:

a callable object

Throws:

NullPointerException - if task null

callable

```
public static Callable<Object> callable(Runnable task)
```

Returns a Callable object that, when called, runs the given task and returns null.

Parameters:

task - the task to run

Returns:

a callable object

Throws:

NullPointerException - if task null

callable

```
public static Callable<Object> callable(PrivilegedAction<?> action)
```

Returns a Callable object that, when called, runs the given privileged action and returns its result.

Parameters:

action - the privileged action to run

Returns:

a callable object

Throws:

NullPointerException - if action null

callable

```
public static Callable<Object> callable(PrivilegedExceptionAction<?> action)
```

Returns a `Callable` object that, when called, runs the given privileged exception action and returns its result.

Parameters:

`action - the privileged exception action to run`

Returns:

`a callable object`

Throws:

`NullPointerException - if action null`

privilegedCallable

```
public static <T> Callable<T> privilegedCallable(Callable<T> callable)
```

Returns a `Callable` object that will, when called, execute the given `callable` under the current access control context. This method should normally be invoked within an `AccessController.doPrivileged` action to create callables that will, if possible, execute under the selected permission settings holding within that action; or if not possible, throw an associated `AccessControlException`.

Type Parameters:

`T - the type of the callable's result`

Parameters:

`callable - the underlying task`

Returns:

`a callable object`

Throws:

`NullPointerException - if callable null`

privilegedCallableUsingCurrentClassLoader

```
public static <T> Callable<T> privilegedCallableUsingCurrentClassLoader(Callable<
```

Returns a `Callable` object that will, when called, execute the given `callable` under the current access control context, with the current context class loader as the context class loader. This method should normally be invoked within an `AccessController.doPrivileged` action to create callables that will, if possible, execute under the selected permission settings holding within that action; or if not possible, throw an associated `AccessControlException`.

Type Parameters:

`T - the type of the callable's result`

Parameters:

`callable - the underlying task`

Returns:

`a callable object`

Throws:

`NullPointerException - if callable null`

`AccessControlException - if the current access control context does not have permission to both set and get context class loader`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.